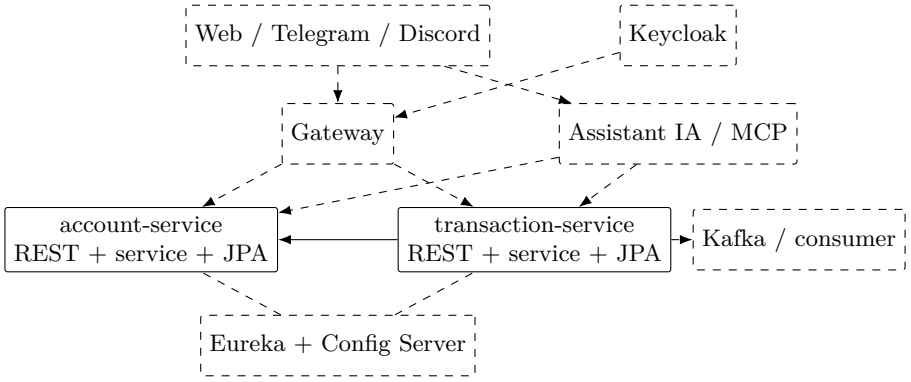
Épreuve technique de sélection

Rapport d'avancement — plateforme bancaire

Youssef Rahioui | 30 octobre 2026

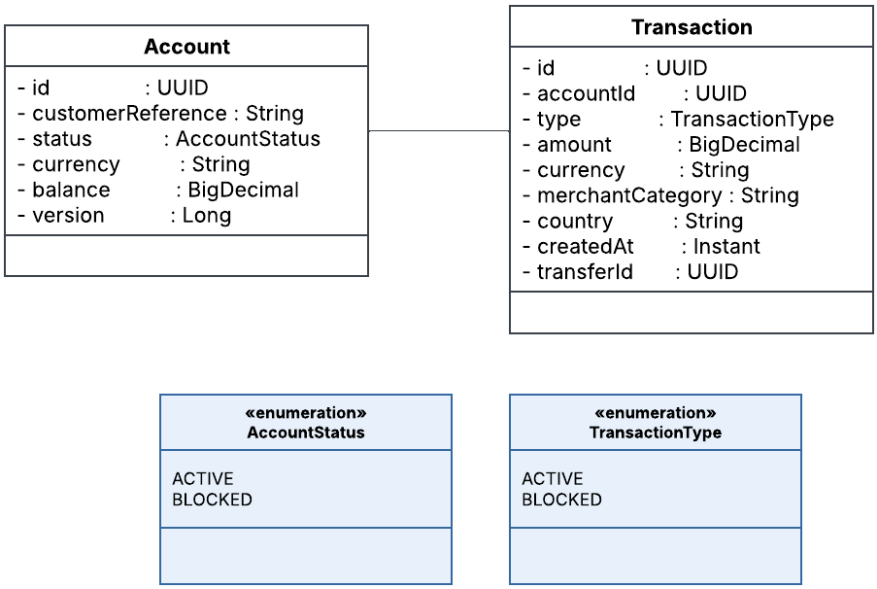
1. Périmètre et architecture (parties A et B)

Le dépôt Maven `bank` contient deux modules : `account-service` et `transaction-service`. Les entités, dépôts JPA, services métier et contrôleurs REST sont présents. Le dessin distingue le code présent des composants prévus.



Légende : boîtes et liaison pleines = code présent (Feign pour la liaison) ; pointillés = éléments prévus. Resilience4J n'est pas implémenté.

Diagramme de classes (fourni avec le projet). Le lien entre les entités est logique, par `accountId`, et non une association JPA.



Note : dans le code, `TransactionType` vaut `DEBIT/CREDIT` (et non `ACTIVE/BLOCKED` comme sur l'image). La liaison dessinée n'est pas une association JPA. Le solde est en `BigDecimal` dans le code.

2. Réalisation et vérification (parties C à G)

Partie	Constat dans le dépôt
C — métier	API REST comptes : création, consultation, solde, débit, crédit. API REST transactions : création, consultation, liste par compte, virement. Les services contrôlent montant, devise, statut et solde ; `AccountClient` relie les modules via OpenFeign. Pas de GraphQL, MCP ni Resilience4J.
D — technique	Clients Config et Eureka déclarés dans les POM ; import Config optionnel. Plafond `bank.transaction.max-amount=1000` défini localement dans `transaction-service`. Aucun serveur Config, Eureka ou Gateway.
E — agent	Aucun assistant Spring AI ni intégration Telegram/Discord ou test de chatbot.
F — Kafka	Aucun producteur, consumer ou événement `TransactionCreated` ; topic `bank.transactions.created` non utilisé.
G — client	Aucun client web ou mobile présent.

Séparation des couches. Les deux modules possèdent `model/`, `repository/`, `service/` et `web/rest/`. Le module de transactions ajoute `client/` (Feign). Aucun `web/graphql/` ni `mcp/` ; les contrôleurs REST appellent la couche service.

Tests exécutés. La commande `mvn test -q` passe depuis la racine : tests de démarrage, tests de services et tests des contrôleurs REST dans les deux modules. Les tests ne constituent pas une démonstration d'une communication réelle entre les services déployés.

3. Détection de fraude (partie H)

Aucun fichier `bank_transactions.csv`, notebook ou script Python n'est présent. Aucune métrique de modèle n'est donc disponible. Protocole attendu : séparation stratifiée avant l'ajustement du pipeline (imputation, encodage, normalisation si nécessaire), exclusion des identifiants, apprentissage sur l'entraînement seul, évaluation sur le test par précision et rappel de la fraude, F1, matrice de confusion et average precision. Le seuil ne doit pas être choisi sur le test.

— **Accuracy :** une classe rare peut être ignorée tout en donnant un score global élevé.
— **Compromis :** plus de détections peuvent accroître les alertes légitimes (faux positifs) ; moins d'alertes peuvent laisser passer des fraudes.
— **Seuil plus bas :** rappel potentiellement plus élevé, mais davantage de faux positifs et souvent une précision moindre.
— **Validation temporelle :** elle reproduit une prédiction sur le futur et réduit les fuites liées aux habitudes et à l'évolution des comportements.

4. Exploitation et livraison (parties A et I)

État : aucun Dockerfile, `compose.yaml` ou workflow CI/CD n'est présent dans ce dépôt. Aucun déploiement ou publication d'image n'est démontré. Le dépôt local contient des commits ; sa visibilité GitHub, la feuille Google et le dépôt Classroom ne sont pas vérifiables à partir des fichiers locaux.

Pipeline envisagé : compilation Maven → tests → construction des images Docker → publication au registre → déploiement depuis une version étiquetée.

— **Secrets / configuration :** jetons, mots de passe et clés dans les secrets CI ou un gestionnaire dédié ; paramètres non sensibles (par exemple plafond) dans la configuration versionnée.
— **Santé :** démarrage vérifié avant le trafic ; readiness pour accepter les requêtes ; liveness pour redémarrer un processus bloqué (Actuator peut fournir les sondes).
— **Observabilité :** logs structurés avec identifiant de corrélation, métriques de latence/erreurs et traces distribuées entre gateway, services et Kafka.
— **Rollback :** redéployer l'image précédemment validée et revenir à la configuration correspondante ; privilégier les migrations de données compatibles.

Bilan. Les opérations REST, la logique métier et le client Feign sont présents ; les tests Maven passent. Il reste à vérifier l'intégration en exécution et à réaliser les autres composants demandés.